# Architectural Design & Implementation Document

## 1. Specification Analysis & Requirements Breakdown

*(Official Project Specification)*

This specification is related to a Linux Kernel Module (LKM) implementing a system call throttling mechanism. The LKM should offer a device driver for supporting the registration/deregistration of:

- program names (executable names)
- user IDs
- syscall numbers (according to x86-64 specification)

Such registration can be supported via the ioctl(...) system call. The registered syscall numbers are used to indicate that the corresponding system calls can be critical for various aspects, like scalability, performance or security.

Each time one of the registered syscall is invoked by a program that is registered by the device driver or by a user (effective user-ID) that is registered by the device driver, the real execution of the system call needs to be controlled by a **monitor** offered by the LKM.

In particular, the LKM monitor can be configured to indicate what is the maximum number MAX of registered syscalls that can be actually invoked in a wall-clock-time window of 1 second by the registered program-names/user-IDs. If the actual volume of invocations per wall-clock-time unit exceeds MAX, then the invoking threads need to be temporarily blocked, thus preventing their actual syscall invocation.

The syscalls managed by the LKM can be of whatever nature. Hence they can be either blocking or non-blocking.

The device driver also needs to offer the possibility to verify at each time instant what are the registered program names, user-IDs and syscall numbers. Also, the update of such information can be only carried out by a thread which is running with effective user-ID set to the value 0 (root).

Also, the device driver needs to support the possibility to turn the syscall monitor off/on along time. If it is turned off, no limit on the frequency of registered syscalls invocations per wall-clock-time unit gets applied. Additionally, on the basis of the selected value for MAX, the device driver should also provide data related to:

- The peak delay for the actual execution of an invoked system call, and the corresponding program-name and user-ID
- The average and peak numbers of threads that had to be blocked

It is expected that the user level code for configuring the LKM monitor, and for testing the correct behavior of the developed LKM software is delivered as part of the project.

## 1.1 Triggering Logic Analysis

The specification defines a precise composite condition for the throttling mechanism. The monitor must act as an active filter between the user-space request and the kernel execution.
The activation logic is formalized as follows:

$$Throttle_Check \Leftrightarrow (SyscallID \in S) \wedge ((UserEUID \in U) \vee (ProgramName \in N))$$

Where $S, U, N$ are the sets of registered syscalls, effective User-IDs, and program names respectively.

**Implication**: The system must strictly distinguish between a registered syscall invoked by a generic user (Allowed) and a registered syscall invoked by a monitored user (Throttled). PIDs are excluded as they are not explicitly required by the specification ("program names" != "process IDs"). Filtering non-matching entities ensures that core OS components (e.g., desktop environments relying heavily on getpid) are not inadvertently starved.

## 1.2 The “Wall-Clock” Constraint

The throughput limit is defined within a "*1-second wall-clock window*".

- **Constraint:** Utilizing CPU Cycle counters (TSC) is unreliable for wall-clock time due to Dynamic Voltage and Frequency Scaling (DVFS) and deep sleep states (C-states). Using High-Resolution Timers (hrtimer) on every syscall would introduce catastrophic overhead in the kernel's hot-path.
- **Solution:** We rely on the Kernel Timekeeping subsystem. Since the window reset (1 second) does not require nanosecond precision, jiffies provides the most efficient, low-overhead time reference. High-resolution timestamps (ktime_get()) will be used strictly for calculating the "Peak Delay" statistics without slowing down the fast-path.

## 1.3 Identity Management

- **Program Names:** The kernel stores the executable name in *current->comm* (limited to TASK_COMM_LEN, typically 16 chars). We use this for lookups. Although the specification informally refers to "executable names", in-kernel matching is implemented against current->comm, i.e. the process command name maintained by the kernel task structure.
- **User IDs:** The specification mandates checking the **Effective User-ID (EUID)**. This is crucial as it reflects the current privilege set (e.g., during setuid execution), distinct from the Real UID.
- **Syscall Numbers:** On x86-64, the syscall number is passed in the RAX register. However, upon entering the kernel dispatcher, regs->ax is proactively overwritten with -ENOSYS to prepare a default failure return value. Accessing the hook array via regs->ax would lead to

negative index out-of-bounds access and Kernel Panics. Thus, the LKM strictly reads regs->orig_ax to safely resolve the original syscall number.

# 2. Architectural Decisions

## 2.1 Interception Mechanism: Ftrace with IPMODIFY

Initially, direct Syscall Table Modification (via CR0 WP disabling) was explored to demonstrate deep kernel memory manipulation. However, on modern bare-metal x86-64 kernels (5.15+), compiler optimizations such as Static/Direct Calls and Retpolines often bypass the sys_call_table, rendering traditional pointer hijacking ineffective and unstable.

- **Decision**: We adopted Ftrace with IPMODIFY (Function Tracer).
- **Justification**: Ftrace is the standard, production-grade Linux tracing API. By registering an ftrace_ops structure with the FTRACE_OPS_FL_IPMODIFY flag, the LKM safely hijacks the instruction pointer (regs->ip) at the very beginning of the target __x64_sys_* stub. This provides robust and reliable interception on the target kernel configuration, while avoiding the fragility of direct syscall table patching on modern systems.

## 2.2 Data Structures: The Scalability Challenge

The monitor runs in the "Hot Path" of the kernel. A linear search O(N) for rules would degrade system performance unacceptably.

- **Decision:** We utilize a Kernel Hash Table (<linux/hashtable.h>).
- **Hash Function:** Integer hashing for EUID/Syscall IDs; String hashing for Program Names.
- **Complexity:** Average case lookup is O(1).

## 2.3 Concurrency Control: RCU(Read-Copy-Update)

To ensure the module's scalability, we use the RCU algorithm for state access:

- **Readers (Syscalls):** Use RCU primitives (rcu_read_lock). They can look up rules concurrently in the fast-path without taking locks and without blocking.
- **Writers (Configuration):** Protected by a Mutex. Rule updates are serialized by the mutex, while readers use RCU to perform lock-free lookups in the hot path. Removed rules are reclaimed through deferred RCU destruction, preventing readers from observing freed memory.

### 2.4 Throttling Logic & Synchronous Guard

- **Global Limit:** Since the MAX parameter applies to the aggregate volume of registered calls system-wide, we employ a global atomic64_t counter. Strict correctness was prioritized over approximate per-cpu counters.
- **Race Condition Mitigation (Double Reset):** A critical concurrency issue exists where multiple threads might cross the 1-second boundary simultaneously. The reset logic is protected by a lightweight Spinlock. This ensures only the *first* thread crossing the boundary performs the reset.
- **Reentrancy Protection (Synchronous Guard):** To prevent infinite recursion caused by Ftrace intercepting the original call made by our own wrapper, we utilize a Synchronous Per-CPU Guard (sc_in_hook). The guard is activated via preempt_disable() *only* during the exact microsecond the original function is invoked. Crucially, preemption is re-enabled before the thread sleeps in the wait queue, rendering the module entirely Migration-Safe and immune to "Scheduling while atomic" deadlocks.

# 3. Implementation Details

## 3.1 Global vs Per-CPU State

To balance consistency and performance:

- **Throttling Counter:** Global (atomic64_t) to ensure strict adherence to the MAX limit.
- **Statistics:** Per-CPU Variables (DEFINE_PER_CPU). Metrics like blocked_count are updated locally by each core to avoid cache-line bouncing. The total is aggregated dynamically during the IOCTL call.
- **Peak Delay:** Stored in a shared sc_peak_record structure protected by an IRQ-safe spinlock, keeping track of delay_ns, euid, and comm.

## 3.2 Blocking Mechanism & Anti-UAF Protection

Threads exceeding the MAX limit yield the CPU via wait_queue_head_t.

- **State:** Threads enter TASK_INTERRUPTIBLE using wait_event_interruptible_timeout.
- **Wake-up:** A wake_up_all() is triggered by the lazy reset logic when the new second begins, or when the monitor is disabled.
- **Safe Unloading (Anti Use-After-Free):** Removing the module while threads are sleeping in its wait queue or while a hook is still being observed by concurrent readers could lead to invalid memory access. The final implementation combines an active_threads barrier, synchronous ftrace unregistration, and RCU grace periods. This ensures that no new callbacks enter the wrapper during teardown and that previously entered readers have safely completed before hook memory is reclaimed.

## 3.3 Identity Management (String Handling)

Matching program names is expensive (strcmp).

- **Optimization:** The Hash Table lookup first computes the Hash of current->comm. strcmp is performed *only* if the hash matches (collision resolution), ensuring near O(1) performance for name verification.

# 4. User-Space Interface (API)

Communication is handled via a Character Device Driver (/dev/sc_throttler) supporting ioctl. Operations are restricted to EUID 0 (root).

| IOCTL Command | Arguments | Description |
|---|---|---|
| IOCTL_ADD_RULE | struct sc_conf * | Registers a Syscall, EUID, or Program Name. |
| IOCTL_DEL_RULE | struct sc_conf * | Removes a specific registered rule. |
| IOCTL_SET_MAX | unsigned long | Sets the global Throughput limit (calls/sec). |
| IOCTL_SET_ONOFF | int | Atomically enables/disables the monitor (0 = OFF, 1 = ON). |
| IOCTL_GET_STATS | struct sc_stats * | Returns aggregated statistics (Peak delay, Avg Blocked, Peak Blocked). |
| IOCTL_LIST_RULES | char* | Serializes all active rules to user-space. |

# 5. Correctness & Security Testing

## 5.1 Functional Verification

- **Rule Enforcement & Independence:** Demonstrated that unregistered users and processes are not throttled, while registered targets are throttled according to the configured MAX limit within the module's jiffies-based 1-second enforcement window.
- **Wall-Clock Precision:** Validation showed that non-throttled calls complete immediately, whereas calls exceeding MAX are delayed until the current jiffies-based 1-second window expires. This provides an efficient wall-clock approximation suitable for the kernel hot path.
- **Privilege Escalation:** Attempting configuration via non-root user correctly yields -EPERM.

## 5.2 Concurrency & Stress Tests

- **Pthread Barrier (Burst Simulation):** 50 threads were aligned through pthread_barrier_wait() to generate a highly synchronized burst of concurrent syscall invocations. The Global Atomic counter and Spinlock correctly handled simultaneous boundary crossings, preserving a consistent global reset behavior under burst contention.

## 5.3 Edge Cases

- **Dynamic Identity Camouflage:** Verified using prctl(PR_SET_NAME) to dynamically change a process's name at runtime, confirming that the kernel's hash lookup accurately detects and throttles the newly assumed identity.
- **Hot-Unplug Survival:** Stress-tested rmmod during active throttling. The final teardown protocol first disconnects ftrace callbacks, then waits for pre-existing readers and active wrapper instances to drain before reclaiming memory. This prevents teardown races and use-after-free conditions during module removal